
Cyberpunk Drone Delivery Simulator

Project Report

Browser-Based 3D Game using WebGL & JavaScript

Submitted by	Md. Zahidul Islam Mehedi
Technology	WebGL, Three.js, JavaScript
Platform	Web Browser (ES Modules)
Date	May 29, 2026

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Motivation	1
1.3	Objectives	1
1.4	Scope and Constraints	2
2	Project Management Plan	3
2.1	Development Methodology	3
2.2	Milestone Schedule	3
2.3	Technology Stack	3
2.4	Risk Management	4
3	Requirement Specifications	5
3.1	Functional Requirements	5
3.2	Non-Functional Requirements	6
4	System Architecture	7
4.1	High-Level Architecture	7
4.2	Rendering Pipeline	7
4.3	Use-Case Diagram	8
4.4	Activity Diagram	9
5	Methodology	10
5.1	Modular ES-Module Design	10
5.2	Drone Physics Model	10
5.3	Weather State Machine	11
5.4	Scoring Algorithm	11
5.5	Collision Detection	12
6	Implementation	13
6.1	Project File Structure	13
6.2	Core Subsystems	13
6.2.1	City Generator (<code>src/scene/</code>)	14
6.2.2	Obstacle System (<code>src/obstacles/</code>)	14
6.2.3	UI System (<code>src/ui/</code>)	14
6.2.4	Tactical Minimap	16
6.2.5	Game Over Statistics	17
6.3	GLSL Shader Design	17
6.4	Key Implementation Decisions	17
6.5	Screenshot and Gameplay View	18

7	Conclusion	19
7.1	Summary	19
7.2	Challenges Encountered	19
7.3	Future Work	20
7.4	Final Remarks	20

List of Figures

4.1	High-Level System Architecture — subsystems managed by <code>main.js</code>	7
4.2	Four-stage rendering pipeline per frame	7
4.3	Use-Case Diagram	8
4.4	Activity Diagram — Full Game Flow	9
6.1	High-level module directory structure (reduced view)	13
6.2	Procedural city generator output — buildings, road grid, and neon accents	14
6.3	Obstacle types: flying cars (Bézier paths), electric wires, and animated billboards	14
6.4	Start screen with neon-themed menu and tutorial panel	14
6.5	Main gameplay view — HUD, drone, and cityscape	15
6.6	Enter Pause Screen	15
6.7	Tutorial Panel	15
6.8	Tactical minimap showing drone (cyan), pickup (green), and drop-off (red) markers	16
6.9	Game Over screen with detailed session statistics	17
6.10	Full gameplay view — drone navigating neon city with active HUD	18

List of Tables

2.1	Development Milestones	3
2.2	Technology Stack	3
2.3	Risk Register	4
3.1	Functional Requirements	5
3.2	Non-Functional Requirements	6
5.1	Drone Physics Constants	10
5.2	Weather States and Parameters	11
5.3	Damage Table by Obstacle Type	12

Chapter 1

Introduction

1.1 Project Overview

The **Cyberpunk Drone Delivery Simulator** is a fully browser-based 3D video game that places the player in the cockpit of an autonomous delivery drone navigating a procedurally generated neon-lit megacity. Players pick up packages from rooftop landing pads and race to drop-off destinations before per-wave timers expire, while dodging flying cars, electric wires, billboard structures, and dynamic weather.

The game is built entirely with **native Web APIs**: a custom WebGL 2 renderer handles geometry, shading, and post-processing, while Three.js is layered for the skybox only—sharing the same canvas context. No build tools, bundlers, or server-side components are required; the project runs by serving static files over HTTP.

1.2 Motivation

Modern web browsers expose powerful GPU compute through WebGL, yet most web games rely on third-party engines such as Unity or Babylon.js. This project demonstrates that a rich real-time 3D experience—including custom GLSL shaders, particle systems, physics simulation, and multi-layer UI—can be authored from scratch in pure JavaScript without sacrificing quality or performance.

The cyberpunk aesthetic was chosen to provide a demanding rendering environment: dense neon geometry, overlapping transparent effects, and dynamic lighting all stress the renderer and highlight the capabilities of the custom pipeline.

1.3 Objectives

- Implement a complete 3D game loop using raw WebGL without a game engine.
- Design a modular, maintainable ES-module codebase spanning 60+ source files.
- Build a procedural city generator that creates a unique skyline on every run.
- Develop a drone physics model with realistic momentum, drag, and wind interaction.
- Create a multi-layer UI system (HUD, minimap, menus, tutorial) using the DOM.
- Implement a dynamic weather state machine affecting gameplay and scoring.

1.4 Scope and Constraints

The project is scoped to a single-player, single-session browser game. It targets desktop browsers with WebGL 2 support (Chrome 80+, Firefox 75+, Edge 80+, Safari 15+). Performance is tuned for 60 fps at 1080p on mid-range hardware. Audio is initialised lazily on first user interaction to comply with browser autoplay policies.

Chapter 2

Project Management Plan

2.1 Development Methodology

The project followed an iterative, milestone-driven approach. Core subsystems were built and verified in isolation before integration. Each subsystem corresponds to a source directory and was developed in dependency order: rendering primitives first, then scene geometry, drone physics, game logic, and finally UI layers.

2.2 Milestone Schedule

Table 2.1: Development Milestones

Phase	Milestone	Deliverable	Status
1	WebGL bootstrap	Canvas, context, shader compile	Complete
2	Scene geometry	Cube geometry, camera, renderer	Complete
3	City generation	Procedural buildings, road grid, skybox	Complete
4	Drone system	Physics, controller, collision AABB	Complete
5	Game logic	Missions, scoring, wave management	Complete
6	Obstacles	Flying cars, wires, billboards	Complete
7	Weather	Rain, fog, wind, sky blending	Complete
8	UI & Audio	HUD, minimap, menus, Web Audio	Complete
9	Post-processing	Bloom pipeline, composite pass	Complete

2.3 Technology Stack

Table 2.2: Technology Stack

Technology	Version	Purpose
WebGL 2	Native browser	3D rendering, shaders
Three.js	r160	Skybox (shared GL context)
gl-matrix	3.4.3	Matrix / vector math
Web Audio API	Native browser	Sound effects
ES Modules	Native browser	Code organisation
Google Fonts	CDN	Orbitron, Share Tech Mono

2.4 Risk Management

Table 2.3: Risk Register

Risk	Severity	Mitigation
Shared WebGL context corruption	High	Three.js <code>autoClear=false</code> ; explicit state restoration
Browser autoplay policy blocking audio	Medium	Lazy <code>AudioContext</code> init on first user gesture
RAF double-tick on restart	Medium	Cancel handle before rescheduling
wires / billboards invisible collision	Medium	Lower speed threshold in damage table
localStorage quota exceeded	Low	Score-only storage; no bulk data

Chapter 3

Requirement Specifications

3.1 Functional Requirements

Table 3.1: Functional Requirements

ID	Requirement	Description
FR-01	Drone Control	Player controls drone via keyboard: thrust, strafe, ascend, descend, yaw.
FR-02	Mission System	Game spawns pickup and drop-off zones on tall buildings; drone must visit both.
FR-03	Wave Progression	Each completed wave increments parcel count (capped at 8) and wave number.
FR-04	Scoring	Score is computed from base delivery, time bonus, package condition, clean-run, weather, and streak multiplier.
FR-05	Health System	Drone has 100 HP; collisions deduct HP per damage table; zero HP ends the game.
FR-06	Weather System	Weather cycles through Clear, Drizzle, Storm, and Fog states with smooth blending.
FR-07	Obstacles	Flying cars on Bézier paths, electric wires, and billboards form dynamic hazards.
FR-08	Minimap	Canvas-based tactical map shows drone position and active mission markers.
FR-09	Pause / Resume	<code>Esc</code> / <code>P</code> toggles pause; rendering continues while paused.
FR-10	High Score	Best score persists across sessions via <code>localStorage</code> .

3.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

ID	Requirement	Description
NFR-01	Performance	Maintain 60 fps at 1080p on mid-range GPU.
NFR-02	Compatibility	Run on Chrome 80+, Firefox 75+, Edge 80+, Safari 15+.
NFR-03	No Build Step	Deployable by serving static files; no transpilation required.
NFR-04	Modularity	Each subsystem isolated in its own ES-module file.
NFR-05	Accessibility	HUD readable at system default font scale; key bindings shown in tutorial.

Chapter 4

System Architecture

4.1 High-Level Architecture

The system is divided into nine subsystems, each mapped to a source directory. `main.js` acts as the composition root, instantiating subsystems, wiring callbacks, and driving the single `requestAnimationFrame` loop.

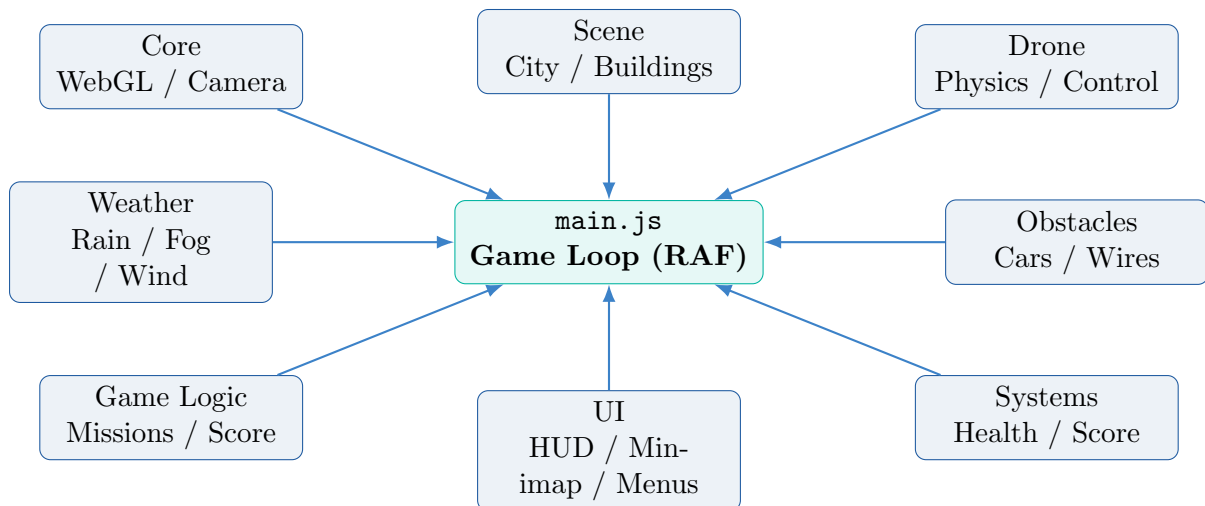


Figure 4.1: High-Level System Architecture — subsystems managed by `main.js`

4.2 Rendering Pipeline

The render pipeline proceeds in four stages each frame, as illustrated in Figure 4.2.

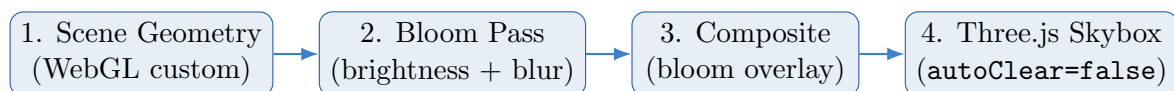


Figure 4.2: Four-stage rendering pipeline per frame

1. **Scene geometry pass** — The custom WebGL renderer draws all city buildings, road grid, ground, obstacles, and the drone mesh using the `city.vert` / `city.frag` GLSL pair.
2. **Brightness extraction** — `brightness.frag` samples the scene framebuffer and outputs only pixels above a luminance threshold, isolating neon glow candidates.

3. **Gaussian blur and composite** — `blur.frag` applies a two-pass Gaussian blur to the brightness texture; `composite.frag` additively blends the result onto the scene, producing the bloom effect.
4. **Three.js skybox** — The Three.js renderer composites the procedural skybox onto the shared canvas without clearing the existing frame (`autoClear = false`).

4.3 Use-Case Diagram

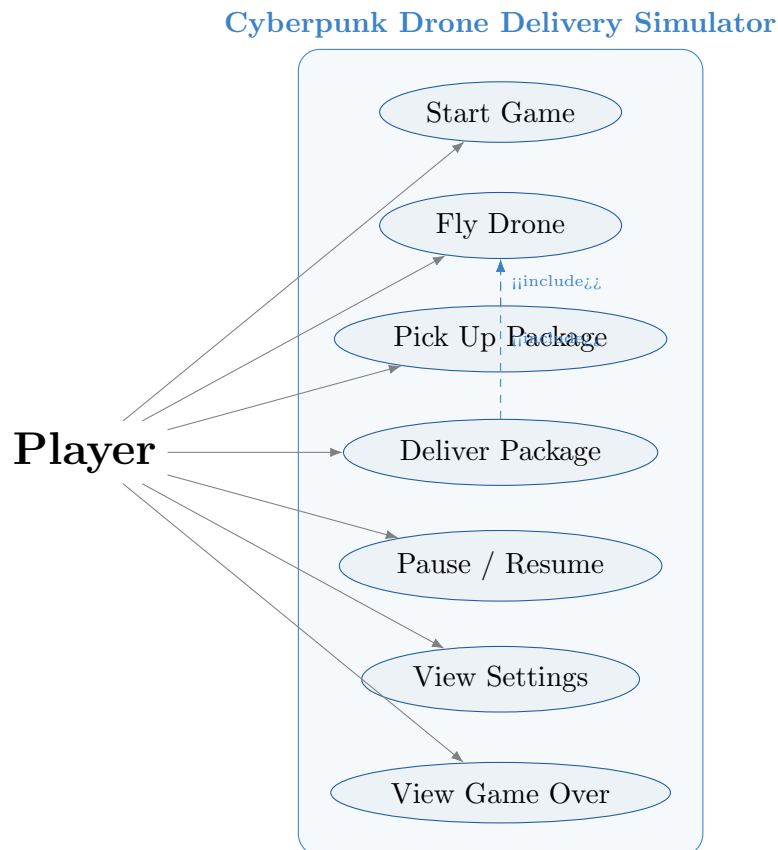


Figure 4.3: Use-Case Diagram

4.4 Activity Diagram

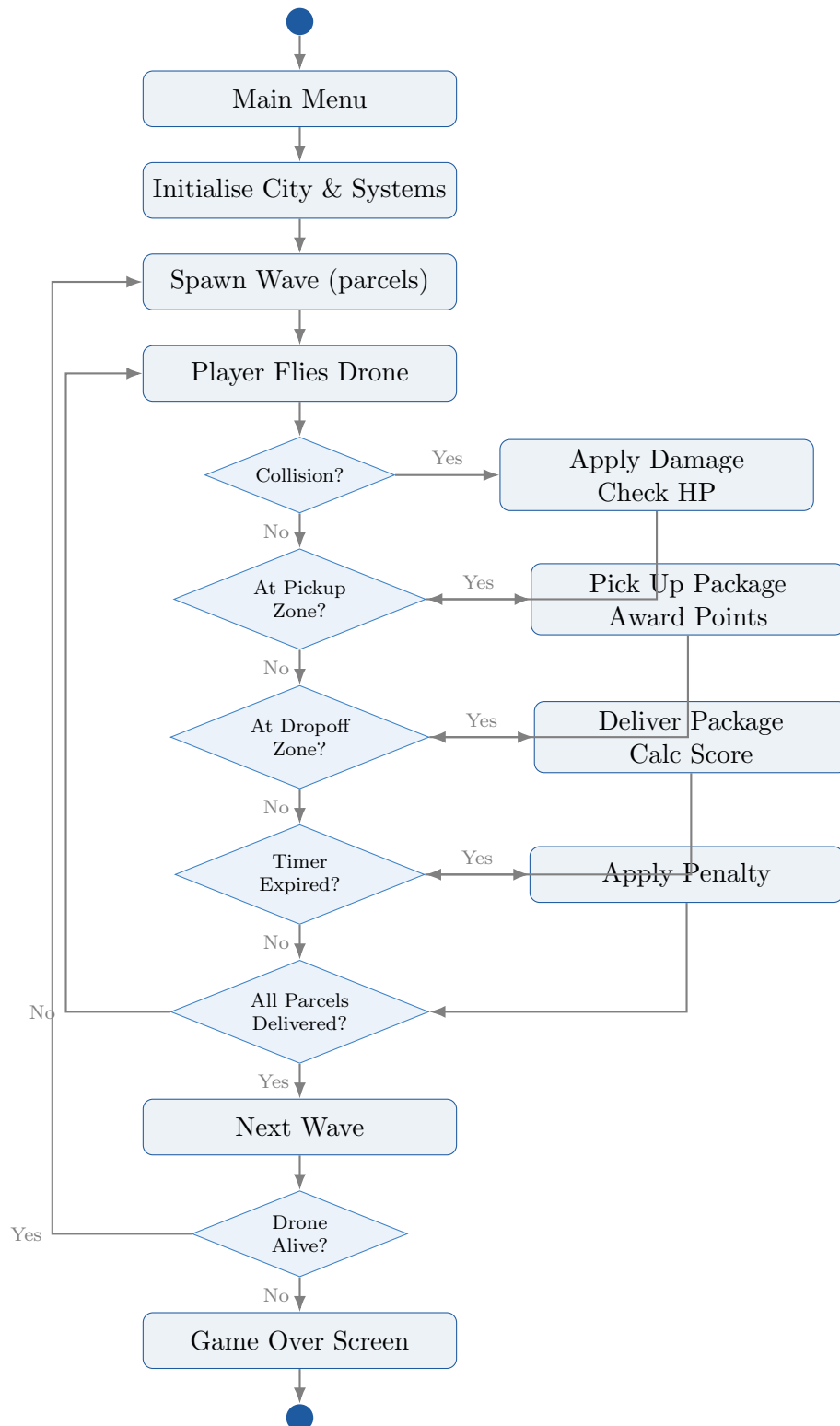


Figure 4.4: Activity Diagram — Full Game Flow

Chapter 5

Methodology

5.1 Modular ES-Module Design

The project adopts a strict ES-module architecture: every class lives in its own `.js` file; there are no global variables. `main.js` is the single composition root that imports all subsystems, constructs them in dependency order, and wires callbacks between them.

```
1 import { initWebGL }      from './src/core/WebGLContext.js';
2 import { Drone }          from './src/drone/Drone.js';
3 import { MissionManager } from './src/game/MissionManager.js';
4 import { WeatherSystem }  from './src/weather/WeatherSystem.js';
5 import { HealthSystem }   from './src/systems/HealthSystem.js';
6 import { ScoreManager }   from './src/systems/ScoreManager.js';
7 import { ScreenManager }  from './src/ui/ScreenManager.js';
```

Listing 5.1: `main.js` — subsystem imports (excerpt)

5.2 Drone Physics Model

The drone is governed by a Newtonian physics model implemented in `DronePhysics.js`. Key constants are shown in Table 5.1.

Table 5.1: Drone Physics Constants

Constant	Value	Description
GRAVITY	9.8 m/s ²	Downward acceleration
THRUST_FORCE	28.0 N	Upward thrust (must exceed gravity)
STRAFE_FORCE	16.0 N	Horizontal thrust
YAW_SPEED	3.2 rad/s	Rotation speed
LINEAR_DRAG	2.2	Velocity damping coefficient
MAX_H_SPEED	38 m/s	Horizontal speed cap
MAX_V_SPEED	24 m/s	Vertical speed cap

Each frame the physics engine reads digital axis inputs, converts them into world-space force vectors (accounting for current yaw), integrates acceleration into velocity, applies drag, and clamps to speed limits:

```

1 // Horizontal forces in world space
2 const worldFwdX = cosY * forwardAxis + sinY * strafeAxis;
3 const worldFwdZ = sinY * forwardAxis - cosY * strafeAxis;
4
5 this.velocity[0] += worldFwdX * STRAFE_FORCE * dt;
6 this.velocity[2] += worldFwdZ * STRAFE_FORCE * dt;
7
8 // Vertical
9 this.velocity[1] += (thrustAxis * THRUST_FORCE - GRAVITY) * dt;
10
11 // Drag
12 this.velocity[0] *= Math.exp(-LINEAR_DRAG * dt);
13 this.velocity[2] *= Math.exp(-LINEAR_DRAG * dt);

```

Listing 5.2: DronePhysics — velocity integration (simplified)

5.3 Weather State Machine

Weather is modelled as a finite-state machine with four states (Table 5.2) and smooth blending over 8 seconds between transitions.

Table 5.2: Weather States and Parameters

State	Rain Particles	Fog Density	Wind (m/s)	Transitions To
CLEAR	0	0.002	0	DRIZZLE, FOG
DRIZZLE	800	0.006	2	STORM, FOG, CLEAR
STORM	4000	0.018	8	DRIZZLE, CLEAR
FOG	200	0.028	1	CLEAR, DRIZZLE

5.4 Scoring Algorithm

The final score for each delivery is computed by `ScoreManager.recordDelivery()`:

$$\text{Score} = (\text{Base} + B_{\text{time}} + B_{\text{pkg}} + B_{\text{weather}} + B_{\text{clean}} + B_{\text{accuracy}}) \times M_{\text{streak}} \quad (5.1)$$

where:

- Base = 600 points for a successful delivery.
- $B_{\text{time}} = 400 \times \frac{t_{\text{remaining}}}{t_{\text{limit}}}$ — time bonus up to 400 pts.
- $B_{\text{pkg}} = 300 \times \frac{\text{condition}}{100}$ — package condition bonus.
- $B_{\text{weather}} \in \{0, 100, 150, 300\}$ — difficulty bonus per weather state.
- $B_{\text{clean}} = 500$ if no collisions occurred during the mission.
- $B_{\text{accuracy}} = 200 \times \max(0, 1 - \frac{v_{\text{approach}} - 3}{9})$ — precision landing.

- $M_{\text{streak}} \in \{1.0, 1.2, 1.5, 1.8, 2.2, 2.6, 3.0\}$ — consecutive-delivery multiplier.

5.5 Collision Detection

Collision uses AABB (Axis-Aligned Bounding Box) tests. Each obstacle type registers a bounding box via `CollisionDetector`. On overlap, `HealthSystem` checks:

1. Whether the impact speed exceeds the per-type minimum damage threshold.
2. Whether the global cooldown timer (0.55 s) has elapsed since the last hit.

If both conditions are met, HP is reduced by the damage table value and the cooldown resets.

Table 5.3: Damage Table by Obstacle Type

Obstacle	HP Damage	Min Speed (m/s)	Pkg Damage Ratio
Building	20	3.0	1.5×
Flying Car	25	2.0	2.0×
Billboard	10	2.0	0.8×
Wire	15	1.5	1.2×
Ground	5	4.0	0.6×

Chapter 6

Implementation

6.1 Project File Structure

The project comprises 67 files across 11 directories. The root entry point is `index.html`, which hosts the WebGL canvas and HUD DOM elements. `main.js` bootstraps all sub-systems asynchronously, loads GLSL shader source via `fetch()`, and enters the render loop.

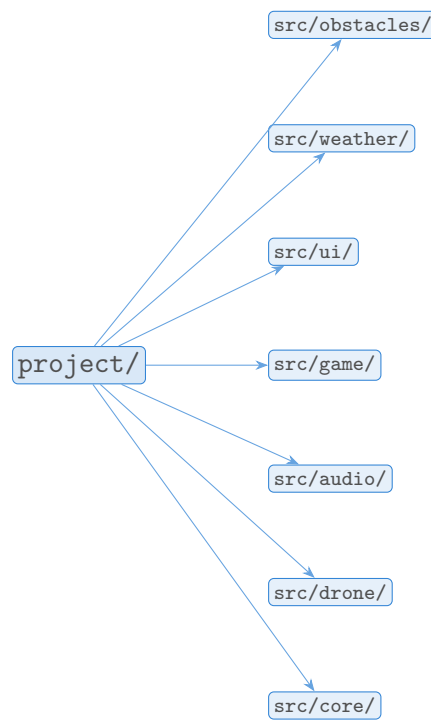


Figure 6.1: High-level module directory structure (reduced view)

6.2 Core Subsystems

6.2.1 City Generator (src/scene/)

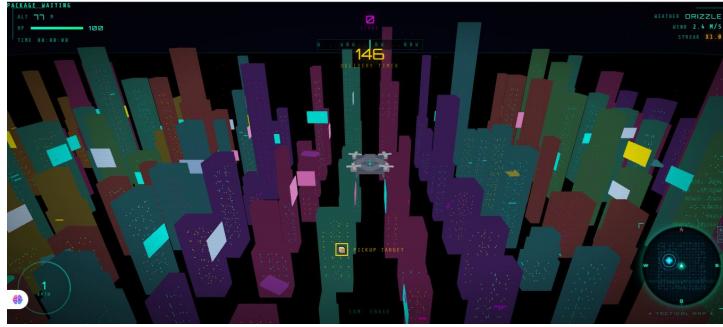


Figure 6.2: Procedural city generator output — buildings, road grid, and neon accents

`RoadGrid.js` overlays a flat grid of road markings. `SkyBox.js` creates a Three.js Mesh with `ShaderMaterial` that renders an atmospheric gradient influenced by the current weather sky-tint uniform.

6.2.2 Obstacle System (src/obstacles/)

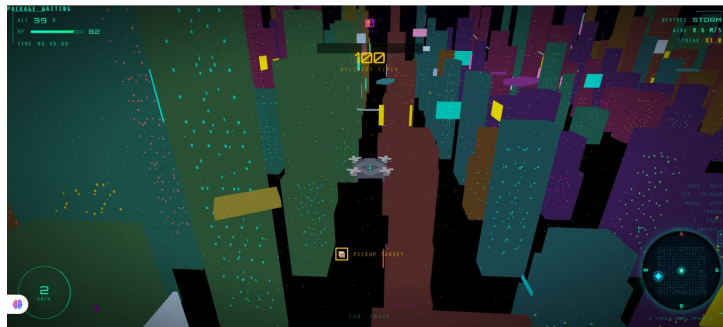


Figure 6.3: Obstacle types: flying cars (Bézier paths), electric wires, and animated billboards

6.2.3 UI System (src/ui/)

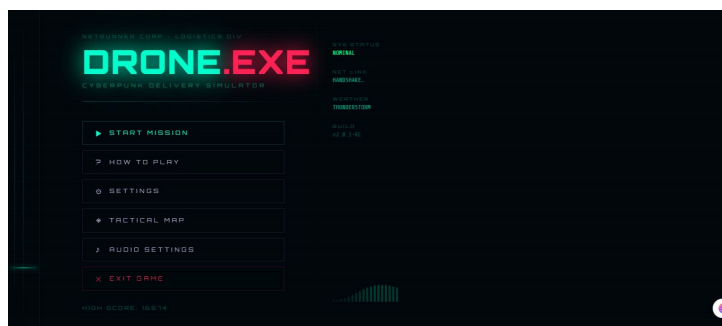


Figure 6.4: Start screen with neon-themed menu and tutorial panel

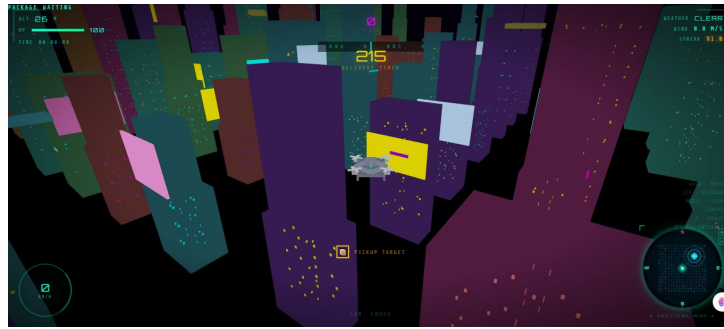


Figure 6.5: Main gameplay view — HUD, drone, and cityscape

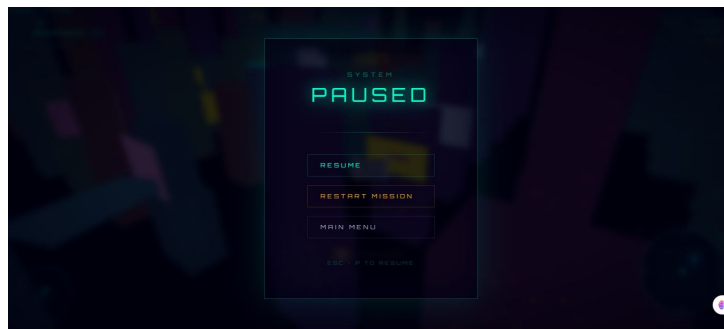


Figure 6.6: Enter Pause Screen

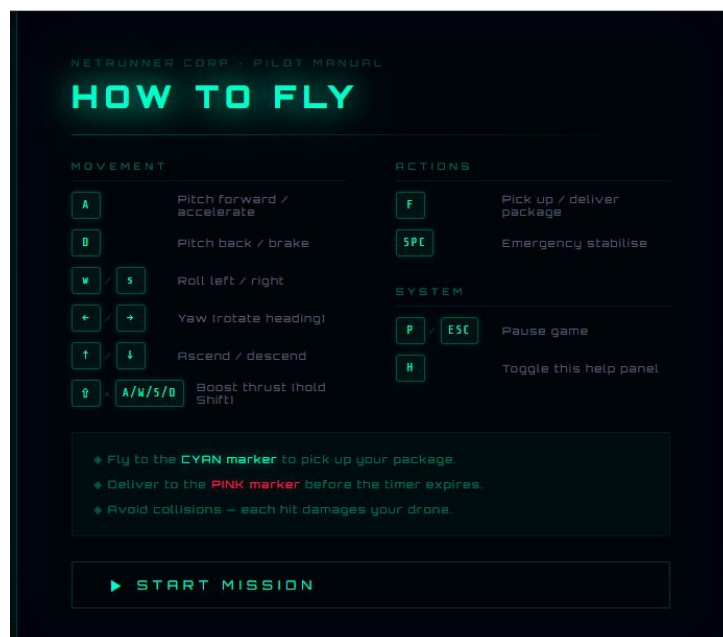


Figure 6.7: Tutorial Panel

`ScreenManager.js` is a 750-line state machine managing four screens: `mainMenu`, `playing`, `paused`, and `dead`. It injects all screen HTML dynamically and routes delegated click events to registered callbacks.

6.2.4 Tactical Minimap

`TacticalMap.js` renders a canvas-based minimap in the HUD corner. `CoordConverter.js` maps 3D world coordinates to 2D minimap pixels using a scaled orthographic projection. `MapMarker.js` draws drone, pickup, and drop-off icons.



Figure 6.8: Tactical minimap showing drone (cyan), pickup (green), and drop-off (red) markers

6.2.5 Game Over Statistics

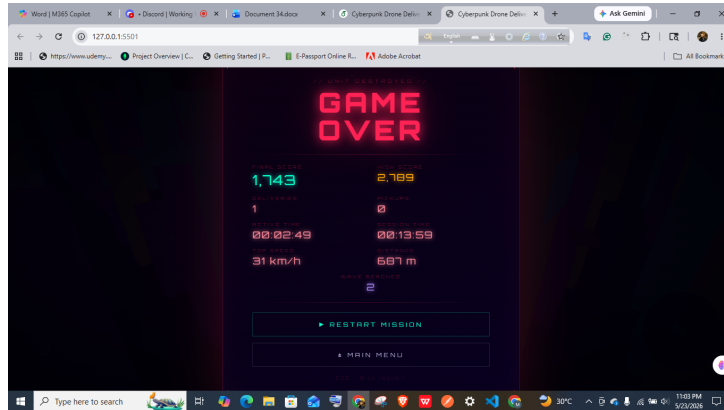


Figure 6.9: Game Over screen with detailed session statistics

When the drone’s HP reaches zero, `GameOverManager.show()` displays a panel with: final score, high score, delivery and pickup counts, active play time, total session time, max speed (converted to km/h), distance flown, and wave reached.

6.3 GLSL Shader Design

The main `city.frag` fragment shader implements:

- Phong lighting with a directional sun and up to four point lights.
- Neon emission pass — objects flagged as neon emit colour regardless of lighting.
- Depth-based exponential fog: $\text{fogFactor} = e^{-d \cdot \text{fogDensity}}$.
- Rain streak simulation via procedural noise on screen-space UVs during storm weather.

6.4 Key Implementation Decisions

No bundler.

Using native ES modules eliminates a build step, making the project immediately runnable. CDN imports for Three.js and gl-matrix are pinned to exact versions for reproducibility.

Shared GL context.

Three.js’s `WebGLRenderer` is constructed with `{ canvas, context: gl }` to share the same context object. Setting `autoClear = false` prevents Three.js from discarding the custom WebGL frame.

Deterministic building seeding.

The LCG seeded from $(x * 31 + z)$ ensures that restarting the game recreates an identical city layout, which is important for the minimap and mission marker positions to remain consistent.

RAF restart safety.

The `rafHandle` variable always holds the current animation frame ID. Before scheduling a new frame, the old handle is cancelled:

```
1 function scheduleLoop(fn) {  
2   if (rafHandle !== null) cancelAnimationFrame(rafHandle);  
3   rafHandle = requestAnimationFrame(fn);  
4 }
```

Listing 6.1: RAF restart guard

Damage cooldown.

A 0.55 s global cooldown prevents a single collision from draining HP multiple times across consecutive frames, which would otherwise make thin obstacles (wires, billboards) disproportionately lethal.

6.5 Screenshot and Gameplay View

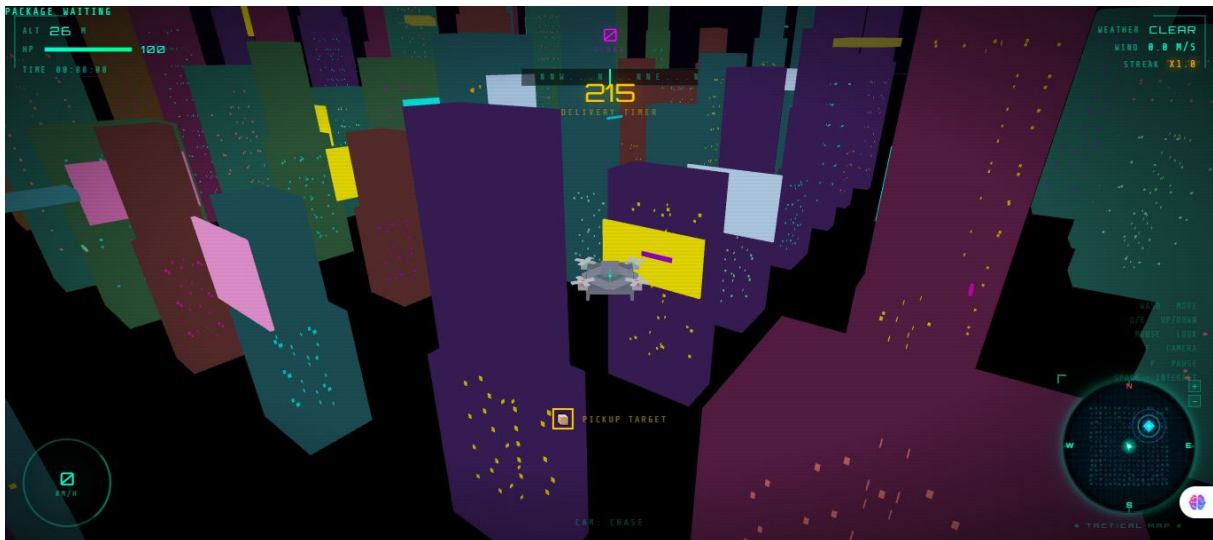


Figure 6.10: Full gameplay view — drone navigating neon city with active HUD

Chapter 7

Conclusion

7.1 Summary

The Cyberpunk Drone Delivery Simulator successfully demonstrates that a feature-rich, real-time 3D game can be built using only browser-native technologies. The project achieves its core objectives: a custom WebGL rendering pipeline with bloom post-processing, a momentum-based drone physics model, procedural city generation, a dynamic weather system, and a fully layered UI—all organised into a maintainable 60+ file ES-module codebase with no build step.

Key technical achievements include:

- A four-stage rendering pipeline (geometry → brightness → blur → composite) delivering convincing neon bloom.
- Shared WebGL context between the custom renderer and Three.js without state corruption.
- A GLSL fragment shader combining Phong lighting, neon emission, exponential fog, and procedural rain streaks in a single pass.
- A comprehensive scoring model rewarding multiple dimensions of pilot skill simultaneously.

7.2 Challenges Encountered

Shared context management.

Preventing Three.js from clearing the custom WebGL framebuffer required careful ordering of draw calls and setting `autoClear = false` before each Three.js render.

Wire collision fairness.

Electric wires are thin and difficult to see. The damage speed threshold was lowered to 1.5 m/s (versus 3.0 m/s for buildings) and the cooldown timer prevents excessive damage accumulation.

RAF ghost loops.

Without explicit cancellation before restarting, multiple animation loops ran simultaneously, causing physics to run at double speed. The `rafHandle` guard eliminated this entirely.

7.3 Future Work

- **Mobile touch controls** — on-screen joystick overlays for tablet play.
- **Night / day cycle** — a slow skybox transition altering lighting and introducing a night-time score bonus.
- **Multiplayer leaderboard** — replace `localStorage` with a serverless score API.
- **Additional drone models** — selectable drone skins with different physics profiles (speed vs. stability).
- **Power-ups** — timed boost pads, shield pickups, and repair kits scattered across rooftops.
- **WebGPU migration** — porting the custom renderer to WebGPU for compute shaders and improved GPU utilisation.

7.4 Final Remarks

The project demonstrates that the modern web platform is a viable target for performance-demanding 3D applications. The absence of a build system, combined with native ES modules and CDN-hosted dependencies, makes the game immediately deployable to any static file host—a significant practical advantage over engine-based alternatives for small-team and solo developers.